

Pre-Workshop Glossary

The Modern Product Owner: Owning the Outcome from Vision to Delivery

Terminology & Definitions — sent in advance to help you get the most from the workshop

How to Use This Glossary

This glossary contains the terms and frameworks you'll encounter in the workshop. Read through it before you arrive — you don't need to memorize anything, but it helps to see the concepts once ahead of time. Bring a copy to the workshop and reference it during activities.

Terms are organized alphabetically. Terms that are central to the workshop (like "Product Owner" or "INVEST") have longer explanations. Supporting terms are brief.

A

Available

[One of the three BE dimensions]

The Product Owner must be present and accessible to the team — not just "in the building" or "reachable by Slack." Practically available means showing up to standups, answering questions in real time, and being the first stop when a team member hits a decision point during sprint work. When the PO is not available, engineers make three kinds of decisions: they wait (losing time), they guess (creating the wrong product), or they build whatever seems most likely (quietly creating the wrong product). Availability is about preventing silence.

B

Backlog / Product Backlog

[The ordered list of everything known to be needed]

The Product Backlog is a living, ordered list of every feature, improvement, bug fix,

research spike, and piece of technical work known to be needed for the product. It's not a wish list. It's not a dump of every idea that came up in a meeting. It's ordered by priority — the top items are ready to build, the items further down are less certain. The PO owns the backlog and its ordering. The team can suggest, but the PO decides what gets built next and why. A clean backlog is a sign of a thoughtful PO. A 400-item backlog with no movement in 6 months is a graveyard, not a backlog.

BDD / Behavior-Driven Development

[An approach to writing acceptance criteria using Given-When-Then (Gherkin)]

A development practice that uses a shared language (Gherkin: Given-When-Then) to describe what features should do. BDD originated to help teams write automated tests, but you can use the Given-When-Then format for acceptance criteria even without automation. BDD makes acceptance criteria testable and unambiguous.

BE

[The first movement of the workshop: Who the PO must be]

Three dimensions that define an effective Product Owner: Available, Knowledgeable, and Empowered. If you have all three, the role works. If you lack one, the role breaks. BE is about identity and presence, not process.

| C

Could Have

[MoSCoW category]

In the MoSCoW prioritization framework, "Could Have" items are nice to build if time permits, but they're not essential for the release. They're typically lower-value or higher-effort than Must or Should haves. Be willing to drop Could haves without guilt if the sprint fills up.

| D

Definition of Done

[The team's quality bar]

A shared agreement between the PO and the team about what it means for a user story to be finished. It typically includes: code written, code reviewed, tests written and passing, documentation updated, the feature deployed to a test environment, and the PO has accepted it. Every story should satisfy the Definition of Done before it's considered complete.

Delighter

[Kano Model category]

A feature users didn't ask for but love when they discover it. Delighters create loyalty and word-of-mouth. Most teams under-invest in delighters because they're hard to predict and easy to skip when a release is tight. One delighter per release often has more impact on customer satisfaction than five performance improvements.

E

Empowered

[One of the three BE dimensions]

The Product Owner must have actual authority to make product decisions — not the title of someone who escalates every call to a steering committee. If every priority decision requires approval, you don't have a PO. You have a backlog coordinator with a fancier title. Empowerment is the authority to decide and live with the consequences.

Epic

[A large unit of work, broken into stories]

An epic is a big feature or capability that's too large to build in a single sprint. It gets broken down into smaller user stories that can fit in a sprint. For example, "User can manage their savings goals" might be an epic that breaks into stories like "User can create a savings goal," "User can view goal progress," "User can delete a goal," etc.

F

Feature

[A capability or piece of functionality]

A feature is something the product does — a capability users interact with. Examples: "Password reset," "Dark mode," "Email notifications." Features change frequently in response to market feedback, user research, and strategic shifts. This is why modern POs commit to themes (which last longer) rather than feature lists (which change constantly).

G

Gap

[The difference between where we are and where we need to be]

The Product Owner role exists to close gaps: the gap between what the business thinks it wants and what users actually need; the gap between long-term vision and next sprint execution; the gap between strategy and delivery. Closing gaps is the core job. Managing artifacts (backlogs, user stories, etc.) is just the tool that helps you do it.

Gherkin / Given-When-Then

[A simple format for writing crystal-clear acceptance criteria]

Gherkin is a structured way to write acceptance criteria so that developers, testers, and stakeholders all understand exactly what "done" means. It uses three simple keywords:

- **Given** — The starting condition or context before the user acts
- **When** — The action the user takes
- **Then** — What happens as a result

Example: Given I am on the login page and have forgotten my password When I click "Forgot Password" and enter my email address Then I receive a password reset email within 2 minutes And I can follow the link to set a new password

Why it matters: Gherkin is the language of Behavior-Driven Development (BDD) and is often used to automate tests (with tools like Cucumber), but it's equally valuable just for writing clearer acceptance criteria in your backlog. When stories have Gherkin acceptance criteria, refinement meetings get shorter, bugs decrease, and the team builds the right thing the first time. Gherkin makes the "Testable" criterion of INVEST concrete and eliminates endless "wait, what did you mean by that?" conversations.

H

Hygiene Factor

[Kano Model category — see "Basic Need"]

I

Independent

[First letter of INVEST]

A user story is independent if it can be built without depending on three other stories first. Independence matters because it gives the team optionality — they can pull stories in different orders, adjust priorities mid-sprint, and parallelize work. If every story depends on the previous story, you've lost flexibility and you'll hit blockers.

INVEST

[A quality bar for user stories — the six letters that define a good story]

A mnemonic for six characteristics every user story should have: - **I**ndependent — can be built without depending on other stories - **N**egotiable — is a conversation starter, not a detailed contract - **V**aluable — delivers value to a user or the business - **E**stimable — the team can size it - **S**mall — fits within a sprint - **T**estable — there's a clear way to know it's done

If a story fails one letter of INVEST, you have a refinement conversation. If it fails three, you don't have a story yet.

| K

Kano Model

[A prioritization framework that classifies features by type of value]

The Kano Model sorts features into three categories based on how they affect customer satisfaction:

- **Basic Needs (Hygiene Factors):** Features users expect. If you have them, users are satisfied. If you don't, they're angry. Doing these well doesn't create love — it just avoids disappointment. Examples: login, password reset, basic security.
- **Performance Features:** More is better. Users notice when you do these well. The stronger your performance features, the happier customers are. Examples: speed, accuracy, customization options.
- **Delighters:** Features users didn't ask for. When executed well, they create delight and loyalty. Examples: unexpected animations, clever shortcuts, surprising integrations.

Most teams over-invest in performance features and under-invest in delighters. The Kano Model helps you balance.

Knowledgeable

[One of the three BE dimensions]

The Product Owner must know the product, the users, the market, and the business well enough to make a defensible decision in 5 minutes — not 5 days.

Knowledgeable means: you know what users do all day; you know what competitors offer; you know what the business needs to achieve; you know enough about the technology to have credible conversations with engineers. This is the dimension new POs often underestimate — they think the role is about managing a process. It's not. It's about carrying enough context that unexpected questions get resolved fast.

| M

Moore's Vision Template

[A structured way to write a product vision]

A template (created by Geoffrey Moore) that forces you to make decisions about your product:

FOR [target customer] WHO [statement of need], OUR [product name] IS A [product category] THAT [key benefit]. UNLIKE [competition], OUR PRODUCT [key differentiation].

Filling this in reveals gaps in your thinking. If you can't complete a blank, you don't have a vision yet — you have a hope.

MoSCoW

[A prioritization framework for a single release]

A way to categorize backlog items for a specific release:

- **Must Have:** Essential for the release. Don't ship without these.
- **Should Have:** Important but not critical. Try to include them, but drop them if time is tight.
- **Could Have:** Nice to have. Only if there's time and capacity.
- **Won't Have:** Explicitly out of scope for this release. (The discipline is in being willing to use this category.)

The key insight: if everything is a Must, you haven't prioritized. You've just listed hopes.

N

Negotiable

[Second letter of INVEST]

A user story is negotiable — it's a conversation starter, not a detailed specification. The story provides enough context for the team to start building, but the exact details emerge through conversation. If a story reads like a legal contract with no room for discussion, it's not negotiable — it's over-specified.

O

Outcome

[What the PO owns, not the backlog]

The success or failure of the product. Revenue targets met or missed. User adoption or churn. Customer satisfaction. Time-to-market. The outcome is what you're accountable for. You can be right about the team missing deadlines, or marketing doing a poor job, or stakeholders changing their minds. But the outcome is still

yours. This discomfort is the role.

| P

Pareto Principle / Pareto Rule

[The 80/20 rule for product work]

Roughly 80% of the value a product delivers comes from 20% of the features. The Pareto Principle reminds the PO to find that 20% and protect it. If you're over-investing in features that don't move the needle, you're wasting the team's time. Pareto is the sanity check on your other prioritization frameworks.

Product

[The thing you're building]

The software, application, service, or feature set you're responsible for. In an agile context, the Product Owner owns decisions about what the product is, what problem it solves, who uses it, and what gets built next.

Product Owner

[The role — abbreviated as PO]

The person accountable for the success of a product. The PO doesn't manage the team (that's the Scrum Master). The PO doesn't do all the building (that's the team). The PO owns direction, prioritization, and stakeholder alignment. The PO's job is to close the gap between what the business wants, what users need, and what's technically feasible. The PO is accountable for the outcome.

| R

Refinement / Backlog Refinement

[A Scrum event where the PO prepares the backlog]

A regular meeting (typically 1-2 hours per sprint for a two-week sprint) where the PO and the team review upcoming backlog items, clarify requirements, break down large items into smaller stories, answer questions, and estimate. Refinement ensures the top of the backlog is ready to pull into the next sprint without surprises. Many POs under-refine and then spend sprint planning trying to clarify stories that should have been clarified already.

| S

Should Have

[MoSCoW category]

In the MoSCoW framework, "Should Have" items are important but not essential for the release. You try to include them, but if the sprint fills up or blockers appear, Should haves are the first things you drop.

Small

[Fifth letter of INVEST]

A user story is small if it can be built, tested, and completed within a single sprint (typically 1-2 weeks). If a story is too big to finish in a sprint, it's not a story — it's an epic that needs to be broken down.

Sprint

[A time-boxed period (usually 1-2 weeks) in which the team builds a set of committed work]

The team chooses a set of stories from the top of the backlog, commits to finishing them within the sprint, and works together toward that goal. The sprint provides rhythm: planning at the start, daily standups, review and retrospective at the end. Most agile teams work in two-week sprints, though some use one-week or three-week sprints.

Sprint Planning

[A Scrum event at the start of each sprint]

The team gathers with the PO to agree on what will be built in the upcoming sprint. The PO presents a sprint goal and the prioritized list of stories. The team estimates, discusses dependencies, and pulls stories into the sprint until they've committed to what they think they can complete. A sprint planning meeting typically takes 2-4 hours for a two-week sprint.

Sprint Review

[A Scrum event at the end of each sprint]

The team demos what was built to stakeholders and the PO. The PO gathers feedback, answers questions, and observes how real people react to the features. The goal is to close the feedback loop — not just to show off what was shipped, but to learn whether the team built the right thing.

Sprint Retrospective

[A Scrum event where the team reflects on process]

The team meets without the PO to discuss: What went well? What could we improve? What will we change next sprint? The retrospective is about the team's process, not the product. The PO is invited to listen, but the retrospective is for the team to voice what's working and what's not.

Stakeholder

[Anyone whose support, input, or approval you need]

Executives, customers, sales teams, marketing, customer support, legal, engineering leadership, regulators — anyone outside the Scrum team who influences whether your product succeeds. Stakeholder management is not a soft skill. It's core PO work. You will spend more time on stakeholder communication than on writing user stories.

Story Mapping

[A visualization technique created by Jeff Patton]

A way to lay out user stories by user journey instead of as a flat list. The top row shows the big steps a user takes through the product. Below each step are the stories that enable that step. Story mapping is excellent at showing you what's missing — if a column has no stories, you have a gap. It's also much easier for stakeholders to understand than a flat backlog list.

T

Testable

[Sixth letter of INVEST]

A user story is testable if there's a clear way to know it's done. "The user can reset their password" is testable — you know the story is done when you can follow the reset flow and receive a new password. "The system should be more scalable" is not testable — there's no objective way to know when it's complete.

Theme

[A strategic direction you commit to, not a specific feature]

Instead of committing to specific features (which change), modern POs commit to themes — broader areas of investment. Example: instead of promising "Add SMS notifications, email notifications, and in-app alerts," commit to the theme "Improving how users stay informed." Features within that theme will change as you learn, but the theme stays consistent.

U

User Story

[A brief description of a capability from the user's perspective]

The standard format: "AS A [type of user], I WANT [a capability], SO THAT [I achieve a benefit]."

Example: "AS A budget-conscious parent, I WANT to see my spending by category, SO THAT I understand where my money goes."

A user story is not a detailed specification. It's a conversation starter. The story provides context; the details emerge through discussion with the team. User stories

should satisfy INVEST.

V

Valuable

[Third letter of INVEST]

A user story is valuable if it delivers value to a user or the business. Value doesn't always mean revenue — it might be customer delight, reduced support burden, improved retention, or faster onboarding. If a story doesn't create value for someone, it shouldn't be in the backlog.

Vision

[The answer to "Why does this product exist?"]

A single sentence or short paragraph that articulates the purpose and direction of the product. Vision is not a tagline or a mission statement written two years ago. It's a clear, memorable statement that anyone on the team can repeat and use to make decisions. The Moore Vision Template is one way to write it down.

W

Won't Have

[MoSCoW category]

In the MoSCoW framework, "Won't Have" items are explicitly out of scope for this release. Being willing to say "we're not doing this" is a sign of clarity. If you can't identify what you're not doing, you haven't truly prioritized.

Z

Quick Reference: The Most Important Terms

If you're pressed for time before the workshop, focus on these:

Term	What It Is
Product Owner	You. Accountable for product success. Owns the outcome.
Backlog	Ordered list of everything to be built. You own the order.
User Story	Brief description of a capability. "AS A ... I WANT ... SO THAT

	..."
INVEST	Quality bar: Independent, Negotiable, Valuable, Estimable, Small, Testable.
Gherkin	Format for acceptance criteria: Given-When-Then. Makes stories crystal clear.
MoSCoW	Prioritization: Must, Should, Could, Won't.
Vision	One sentence: why does this product exist?
Roadmap	Now / Next / Later. Commit to themes, not features.
Available	You're present and accessible to the team.
Knowledgeable	You know the product, users, market, and business.
Empowered	You have authority to decide and live with consequences.
Stakeholder	Anyone whose support or approval you need.
Sprint	1-2 week cycle in which the team builds committed work.
Outcome	What the product achieves. Revenue, adoption, satisfaction, speed. You own it.

Before the Workshop

You don't need to memorize this glossary. Just read it through once to get familiar with the language. Bring a copy to the workshop — you'll reference it during the activities. The real learning happens in the room through discussion and practice, not through definitions.

See you there.

Diego Avella
AI Product Manager · Scrum Master

Diego Avella
 AI Product Manager · Scrum Master